

# DUEL MASTERS TCG ENGINE

Game Design Document (GDD) & Full-Stack System Architecture

GODOT 4.X (C# MONO)

.NET 9 (SIGNALR/WEBSOCKETS)

NESTJS ARCHITECTURE READY

SPRING BOOT ARCHITECTURE READY

MASTER DUEL VISUAL STYLE

## 1. Executive Summary & Vision Statement

This Game Design Document outlines the complete technical, visual, and architectural specification for recreating the classic **Duel Masters Trading Card Game** as a modern, high-performance, real-time digital TCG. The project combines **Godot Engine 4.x (C# Edition)** for client rendering and 2.5D visual effects with a high-throughput, authoritative **.NET 9 backend** (utilizing SignalR / WebSockets for real-time state synchronization).

### Client Vision (Godot 4.x)

- 2.5D Dynamic Arena:** Angled board view with dynamic perspective tilt inspired by *Yu-Gi-Oh! Master Duel*.
- High-Impact VFX:** Custom shaders for shield breaking, mana tapping, creature summoning, and civilization energy bursts.
- Tactile Mechanics:** Dynamic card drag-and-drop, curve hand layout, energy counters inspired by *Pokémon TCG*.

### Backend Architecture (.NET 9)

- 100% Authoritative Logic:** Prevent client side anti-cheat manipulation; all game state turns, mana rules, and battles execute on server.
- Protocol Agnostic:** Clean Event-Driven JSON WebSockets contract, allowing seamless future backend swaps to **NestJS** or **Spring Boot**.
- Asset Pipeline:** Data ingestion tool to turn raw image files into structured JSON metadata.

## 2. Core Game Mechanics & Rules Specification

The rules engine strictly enforces authentic Duel Masters gameplay mechanics across all 5 Civilizations. Every turn is governed by a deterministic state machine.

| Civilization | Primary Color   | Core Identity & Mechanics                                                | Signature Keywords                  |
|--------------|-----------------|--------------------------------------------------------------------------|-------------------------------------|
| Light        | Gold / Yellow   | High Defense, untapping creatures, mana recovery, untargetable barriers. | Blocker, Untap, Shield Addition     |
| Water        | Sapphire / Blue | Card draw, hand bouncing, unblockable attacks, spell searching.          | Draw, Return to Hand, Liquid People |
| Darkness     | Shadow Purple   | Creature destruction, hand discard, graveyard reanimation, sacrifice.    | Slayer, Discard, Reanimate          |
| Fire         | Crimson Red     | Aggressive rush, speed attackers, small creature removal, power boosts.  | Speed Attacker, Power Attacker      |
| Nature       | Emerald Green   | Mana acceleration, deck creature search, massive power buffs, evolution. | Mana Boost, Search Deck, Evolution  |

### 2.1 Turn Cycle State Machine

- UNTAP\_PHASE:** All tapped creatures and mana cards owned by the active player return to an Untapped state.
- DRAW\_PHASE:** Active player draws 1 card from their main deck (Player 1 Turn 1 skip applies if using classic competitive rule option).
- MANA\_PHASE:** Active player may select 1 card from hand to deposit face-down (or inverted) into the Mana Zone. Grants +1 available mana matching that card's civilization.

4. **MAIN\_PHASE:** Active player can summon Creatures (paying cost by tapping matching civilization mana) or cast Spells. Evolution creatures require a matching race body in the Battle Zone.
5. **ATTACK\_PHASE:** Active player taps an untapped creature to attack an opponent directly OR attack a tapped enemy creature.
  - *Blocker Step:* Opponent may declare a Blocker creature to redirect the attack.
  - *Battle Step (Creature vs Creature):* Higher power survives; lower power is destroyed to Graveyard. Ties destroy both.
  - *Shield Break Step (Creature vs Player):* Attacker breaks 1 or more shields (Double/Triple Breaker). Defender puts broken shields into hand.
  - *Shield Trigger Window:* Defender reveals any broken shield cards with `Shield Trigger`. Spells/Creatures with Shield Trigger cast IMMEDIATELY for 0 mana before the next attack proceeds.
6. **END\_PHASE:** Trigger cleanups, end-of-turn passive effects reset, turn passes to opponent.

### 3. Asset & Card Metadata Pipeline (Handling Image-Only Assets)

**Asset Integration Solution:** Because your card collection consists of raw `.png` / `.jpeg` images without structured card text or numerical stats, we implement an automated Python Data Ingestion Pipeline. It scans image filenames, matches them with standardized JSON metadata (scraped or defined via OCR), and produces a deterministic `cards.json` database for Godot and .NET.

#### 3.1 Card Database JSON Schema ( `cards.json` )

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "name": { "type": "string" },
      "civilization": { "type": "string", "enum": ["Light", "Water", "Darkness", "Fire", "Nature", "Zero"] },
      "cardType": { "type": "string", "enum": ["Creature", "Spell", "EvolutionCreature"] },
      "manaCost": { "type": "integer", "minimum": 1 },
      "manaNumber": { "type": "integer", "default": 1 },
      "power": { "type": "integer" },
      "race": { "type": "string" },
      "imagePath": { "type": "string" },
      "keywords": {
        "type": "array",
        "items": { "type": "string" }
      },
      "scriptEffectId": { "type": "string" }
    },
    "required": ["id", "name", "civilization", "cardType", "manaCost", "imagePath"]
  }
}
```

#### 3.2 Complete Ingestion Pipeline Script ( `ingest_cards.py` )

```

import os
import json
import glob
from pathlib import Path

# Directory configuration
IMAGE_DIR = "./assets/cards_raw"
OUTPUT_JSON = "./data/cards.json"

DEFAULT_CARD_TEMPLATES = {
    "dm_01_001": {
        "name": "Rothus, the Traveler",
        "civilization": "Fire",
        "cardType": "Creature",
        "manaCost": 4,
        "power": 4000,
        "race": "Armor Dragon",
        "keywords": ["POWER_ATTACKER_2000"],
        "scriptEffectId": "EFFECT_ROTHUS_DESTROY_SELF"
    },
    "dm_01_002": {
        "name": "Aqua Vehicle",
        "civilization": "Water",
        "cardType": "Creature",
        "manaCost": 2,
        "power": 1000,
        "race": "Liquid People",
        "keywords": [],
        "scriptEffectId": "VANILLA"
    },
    "dm_01_003": {
        "name": "Holy Awe",
        "civilization": "Light",
        "cardType": "Spell",
        "manaCost": 6,
        "power": 0,
        "race": "Spell",
        "keywords": ["SHIELD_TRIGGER"],
        "scriptEffectId": "EFFECT_TAP_ALL_ENEMY_CREATURES"
    }
}

def generate_card_database():
    card_database = []
    image_files = glob.glob(os.path.join(IMAGE_DIR, "*.png"))

    print(f"[Pipeline] Found {len(image_files)} card images in {IMAGE_DIR}")

    for filepath in sorted(image_files):
        filename = Path(filepath).stem
        ext = Path(filepath).suffix
        relative_path = f"res://assets/cards/{filename}{ext}"

        # Check if template metadata exists, else assign default structured payload
        meta = DEFAULT_CARD_TEMPLATES.get(filename, {
            "name": filename.replace("_", " ").title(),
            "civilization": "Fire",
            "cardType": "Creature",
            "manaCost": 3,
            "power": 2000,
            "race": "Dragonoid",
            "keywords": [],
            "scriptEffectId": "VANILLA"
        })

```

```

        card_entry = {
            "id": filename,
            "name": meta["name"],
            "civilization": meta["civilization"],
            "cardType": meta["cardType"],
            "manaCost": meta["manaCost"],
            "manaNumber": 1,
            "power": meta["power"],
            "race": meta["race"],
            "imagePath": relative_path,
            "keywords": meta["keywords"],
            "scriptEffectId": meta["scriptEffectId"]
        }
        card_database.append(card_entry)

    os.makedirs(os.path.dirname(OUTPUT_JSON), exist_ok=True)
    with open(OUTPUT_JSON, "w", encoding="utf-8") as f:
        json.dump(card_database, f, indent=2)

    print(f"[Pipeline] Successfully exported {len(card_database)} cards to {OUTPUT_JSON}")

if __name__ == "__main__":
    generate_card_database()

```

## 4. UI/UX & VFX Design Specification (Master Duel Style)

To replicate the immersive feel of modern TCG titles like Yu-Gi-Oh! Master Duel while keeping the fast-paced nature of Duel Masters, the rendering pipeline utilizes a 2.5D perspective viewport with dynamic camera framing.

### Visual Breakdown & Camera Perspective

- **Arena Camera:** Godot `Camera3D` or `Camera2D` with Y-sort tilt angled at 35 degrees towards opponent side. Dynamic FOV punch-in on attack declarations and shield breaks.
- **Hand Rail:** Dynamic arc positioning at screen bottom. Cards scale 1.3x on hover with slight elevation (`Z-Index`), showing glowing valid play outlines (Green = Mana turn play, Blue = Creature summon play).
- **Mana Pool HUD:** Located bottom-left (Player) and top-right (Opponent). Cards fan out with a numeric civilization counter badge overlay (e.g. [Fire: 3/3] [Water: 2/2]).
- **Shield Zone Display:** 5 face-down cards on left side of arena. Glowing aura matching owner's highest civilization concentration.

### 4.1 Shield Break Shader & Particle Pipeline

When a shield is struck during combat, Godot executes a 3-tier sequence:

1. **Freeze Frame & Camera Shake:** Engine pauses state for 0.15s with radial blur post-processing.
2. **Shatter Mesh / Shader:** Shield card dissolves using a fragment noise shader with golden glowing edges.
3. **Shield Trigger Resolution Window:** If card has `SHIELD\_TRIGGER`, the card zooms to screen center with a dramatic light ray particle background, giving defender a clear "ACTIVATE SHIELD TRIGGER?" prompt.

## 5. Godot 4.x Client Architecture (C# Mono)

The client uses Godot 4.x with C# for strong typing, fast execution, and direct shared class definitions with the .NET backend.

### 5.1 Client Scene Graph Architecture

```
RootNode (Node2D/Node3D)
├─ CameraController (Camera2D/Camera3D)
├─ BoardView (Node)
│   └─ BattleZone (Node2D) -> Container for Creature Nodes
│       └─ ManaZone (Node2D) -> Container for Tapped Mana Cards
│           └─ ShieldZone (Node2D) -> Container for 5 Shield Nodes
│               └─ GraveyardZone (Node2D)
├─ HandUI (CanvasLayer)
│   └─ HandContainer (Control) -> Dynamic Card Slot Layout
├─ VFXManager (Node) -> Spawner for Shaders, Explosions & Ray Beams
└─ NetworkManager (Node) -> SignalR / WebSocket C# Client Singleton
```

## 5.2 Godot Card View Node ( CardNode.cs )

```

using Godot;
using System;

public partial class CardNode : Area2D
{
    [Signal] public delegate void CardClickedEventHandler(string cardId);
    [Signal] public delegate void CardHoveredEventHandler(string cardId, bool isHovered);

    [Export] public Sprite2D CardArtwork { get; set; }
    [Export] public Label PowerLabel { get; set; }
    [Export] public Label ManaCostLabel { get; set; }
    [Export] public Control OutlineHighlight { get; set; }

    public string CardId { get; private set; }
    public bool IsTapped { get; private set; } = false;
    private Vector2 _targetPosition;
    private float _targetRotation = 0f;

    public override void _Ready()
    {
        MouseEntered += OnMouseEntered;
        MouseExited += OnMouseExited;
    }

    public void SetupCard(string id, string artworkPath, int power, int manaCost)
    {
        CardId = id;
        PowerLabel.Text = power > 0 ? power.ToString() : "";
        ManaCostLabel.Text = manaCost.ToString();

        var texture = GD.Load<Texture2D>(artworkPath);
        if (texture != null)
        {
            CardArtwork.Texture = texture;
        }
    }

    public override void _Process(double delta)
    {
        // Smooth interpolation for position and rotation lerping
        Position = Position.Lerp(_targetPosition, (float)delta * 12.0f);
        Rotation = Mathf.LerpAngle(Rotation, _targetRotation, (float)delta * 12.0f);
    }

    public void SetTappedState(bool tapped)
    {
        IsTapped = tapped;
        _targetRotation = tapped ? Mathf.DegToRad(90.0f) : 0.0f;
    }

    public void MoveTo(Vector2 newPos)
    {
        _targetPosition = newPos;
    }

    private void OnMouseEntered()
    {
        EmitSignal(SignalName.CardHovered, CardId, true);
        Scale = new Vector2(1.15f, 1.15f);
    }

    private void OnMouseExited()
    {
        EmitSignal(SignalName.CardHovered, CardId, false);
        Scale = Vector2.One;
    }
}

```

```
    }  
  
    public override void _InputEvent(Godot.Object viewport, InputEvent @event, long shapeIdx)  
    {  
        if (@event is InputEventMouseButton mouseBtn && mouseBtn.Pressed && mouseBtn.ButtonIndex ==  
MouseButton.Left)  
        {  
            EmitSignal(SignalName.CardClicked, CardId);  
        }  
    }  
}
```

### 5.3 Godot Board Controller ( BoardController.cs )

```

using Godot;
using System;
using System.Collections.Generic;

public partial class BoardController : Node2D
{
    [Export] public PackedScene CardNodePrefab { get; set; }
    [Export] public Node2D BattleZoneContainer { get; set; }
    [Export] public Node2D ManaZoneContainer { get; set; }
    [Export] public Control HandContainer { get; set; }

    private Dictionary<string, CardNode> _activeCards = new();

    public override void _Ready()
    {
        // Connect network listener events
        NetworkManager.Instance.OnGameStateUpdated += UpdateBoardState;
    }

    public void UpdateBoardState(GameStateDto state)
    {
        // Render Active Player Hand Cards
        foreach (var cardDto in state.ActivePlayerHand)
        {
            if (!_activeCards.ContainsKey(cardDto.InstanceId))
            {
                var newCard = CardNodePrefab.Instantiate<CardNode>();
                HandContainer.AddChild(newCard);
                newCard.SetupCard(cardDto.CardId, cardDto.ImagePath, cardDto.Power, cardDto.ManaCost);
                newCard.CardClicked += (id) => OnHandCardSelected(cardDto.InstanceId);
                _activeCards[cardDto.InstanceId] = newCard;
            }
        }

        // Render Mana Zone Placement & Tapped States
        for (int i = 0; i < state.ActivePlayerMana.Count; i++)
        {
            var manaDto = state.ActivePlayerMana[i];
            if (_activeCards.TryGetValue(manaDto.InstanceId, out var cardNode))
            {
                cardNode.Reparent(ManaZoneContainer);
                cardNode.MoveTo(new Vector2(i * 60.0f, 0));
                cardNode.SetTappedState(manaDto.IsTapped);
            }
        }
    }

    private void OnHandCardSelected(string instanceId)
    {
        GD.Print($"[Client] Card clicked: {instanceId}");
        NetworkManager.Instance.SendPlayCardRequest(instanceId);
    }
}

```

## 6. Authoritative Backend Architecture (.NET 9 + WebSockets)

The backend core is structured following **Clean Architecture** principles. The core domain logic is isolated inside a pure C# assembly (zero third-party dependencies), allowing instant unit testing and seamless transport via SignalR in .NET 9, Socket.io in NestJS, or STOMP in Spring Boot.

### 6.1 Protocol Neutral Event Schema (JSON)

Every client interaction transmits a protocol packet following this standard OpenAPI/AsyncAPI contract:



```
{
  "action": "DECLARE_ATTACK",
  "sessionId": "session_99412",
  "playerId": "player_01",
  "payload": {
    "attackerInstanceId": "inst_creature_44",
    "targetInstanceId": "inst_shield_02",
    "targetPlayerId": "player_02"
  }
}
```

## 6.2 Domain Game Engine State ( `DuelGameState.cs` )

```

namespace DuelMasters.Domain;

public enum TurnPhase
{
    Untap,
    Draw,
    Mana,
    Main,
    Attack,
    End
}

public class CardInstance
{
    public string InstanceId { get; set; } = Guid.NewGuid().ToString();
    public string CardId { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Civilization { get; set; } = string.Empty;
    public string CardType { get; set; } = string.Empty; // Creature, Spell, Evolution
    public int ManaCost { get; set; }
    public int Power { get; set; }
    public bool IsTapped { get; set; } = false;
    public bool HasSummoningSickness { get; set; } = true;
    public List<string> Keywords { get; set; } = new();
}

public class PlayerState
{
    public string PlayerId { get; set; } = string.Empty;
    public List<CardInstance> Deck { get; set; } = new();
    public List<CardInstance> Hand { get; set; } = new();
    public List<CardInstance> ManaZone { get; set; } = new();
    public List<CardInstance> BattleZone { get; set; } = new();
    public List<CardInstance> Shields { get; set; } = new();
    public List<CardInstance> Graveyard { get; set; } = new();
}

public class DuelGameState
{
    public string MatchId { get; set; } = Guid.NewGuid().ToString();
    public PlayerState Player1 { get; set; } = new();
    public PlayerState Player2 { get; set; } = new();
    public string ActivePlayerId { get; set; } = string.Empty;
    public TurnPhase CurrentPhase { get; set; } = TurnPhase.Untap;
    public int TurnCount { get; set; } = 1;
    public bool IsGameOver { get; set; } = false;
    public string WinnerPlayerId { get; set; } = string.Empty;

    public bool PlayCardToMana(string playerId, string instanceId)
    {
        if (ActivePlayerId != playerId || CurrentPhase != TurnPhase.Mana) return false;

        var player = GetPlayer(playerId);
        var card = player.Hand.FirstOrDefault(c => c.InstanceId == instanceId);
        if (card == null) return false;

        player.Hand.Remove(card);
        player.ManaZone.Add(card);
        CurrentPhase = TurnPhase.Main; // Transition to Main Phase after mana deposit
        return true;
    }

    public bool DeclareAttack(string attackerId, string targetId, out string resultSummary)
    {
        resultSummary = "";
    }
}

```

```

        var attackerPlayer = GetPlayer(ActivePlayerId);
        var defenderPlayer = GetOpponent(ActivePlayerId);

        var attackerCard = attackerPlayer.BattleZone.FirstOrDefault(c => c.InstanceId == attackerId);
        if (attackerCard == null || attackerCard.IsTapped || attackerCard.HasSummoningSickness)
            return false;

        attackerCard.IsTapped = true; // Tap attacker

        // Target check: Shield or Creature
        var targetShield = defenderPlayer.Shields.FirstOrDefault(s => s.InstanceId == targetId);
        if (targetShield != null)
        {
            defenderPlayer.Shields.Remove(targetShield);
            defenderPlayer.Hand.Add(targetShield); // Broken shield to hand
            resultSummary = $"SHIELD_BROKEN:{targetShield.InstanceId}";

            if (defenderPlayer.Shields.Count == 0 && attackerPlayer.BattleZone.Any(c => c.InstanceId
== attackerId))
            {
                // Win Condition Check on direct attack with zero shields
                IsGameOver = true;
                WinnerPlayerId = ActivePlayerId;
                resultSummary = "GAME_OVER_VICTORY";
            }
            return true;
        }

        return false;
    }

    public PlayerState GetPlayer(string id) => Player1.PlayerId == id ? Player1 : Player2;
    public PlayerState GetOpponent(string id) => Player1.PlayerId == id ? Player2 : Player1;
}

```

### 6.3 Authoritative SignalR Server Hub (Due1Hub.cs )

```

using Microsoft.AspNetCore.SignalR;
using DuelMasters.Domain;

namespace DuelMasters.Infrastructure;

public interface IDuelClientContract
{
    Task ReceiveGameState(DuelGameState state);
    Task ReceiveActionError(string errorMessage);
    Task TriggerShieldBreakVFX(string shieldInstanceId, bool isShieldTrigger);
    Task AnnounceWinner(string winnerPlayerId);
}

public class DuelHub : Hub<IDuelClientContract>
{
    private static readonly Dictionary<string, DuelGameState> ActiveMatches = new();

    public async Task PlayManaCard(string matchId, string cardInstanceId)
    {
        if (!ActiveMatches.TryGetValue(matchId, out var match))
        {
            await Clients.Caller.ReceiveActionError("Match session not found.");
            return;
        }

        var playerId = Context.ConnectionId;
        bool success = match.PlayCardToMana(playerId, cardInstanceId);

        if (success)
        {
            await Clients.Group(matchId).ReceiveGameState(match);
        }
        else
        {
            await Clients.Caller.ReceiveActionError("Invalid mana play action.");
        }
    }

    public async Task ExecuteAttack(string matchId, string attackerInstanceId, string targetInstanceId)
    {
        if (!ActiveMatches.TryGetValue(matchId, out var match)) return;

        bool success = match.DeclareAttack(attackerInstanceId, targetInstanceId, out string summary);
        if (success)
        {
            if (summary.StartsWith("SHIELD_BROKEN"))
            {
                var shieldId = summary.Split(':')[1];
                await Clients.Group(matchId).TriggerShieldBreakVFX(shieldId, false);
            }

            await Clients.Group(matchId).ReceiveGameState(match);

            if (match.IsGameOver)
            {
                await Clients.Group(matchId).AnnounceWinner(match.WinnerPlayerId);
            }
        }
    }
}

```

## 7. Step-by-Step Implementation Roadmap

**Milestone Execution Path:** Work sequentially through these 5 development phases to bring the game from asset ingestion to polished multiplayer release.

### 1 Phase 1: Data Ingestion & JSON Database

- Execute `ingest_cards.py` on your existing raw `.png` / `.jpeg` card artwork directory.
- Map initial 40 base cards to stats (Name, Civilization, Power, Mana Cost, Keywords).
- Validate generated `cards.json` file structure.

### 2 Phase 2: Offline Domain Rules Machine (C# Library)

- Implement `DuelMasters.Domain` class library with zero engine dependencies.
- Write xUnit unit tests for turn flow: Untap -> Draw -> Mana deposit -> Creature play -> Battle resolution -> Shield breaking.
- Verify Shield Trigger interrupt window logic.

### 3 Phase 3: Godot 4 Client Board Visualizer & Local Sandbox

- Create `CardNode.tscn` with card artwork loader, hover scaling, and 90-degree tap rotation lerp.
- Build 2.5D Arena Layout: Battle Zone grid, Hand curve, Mana bottom-left pool, and 5 Shield cards.
- Connect local game state machine to play a 2-player hotseat match on single PC.

### 4 Phase 4: Authoritative .NET 9 Backend Integration

- Create ASP.NET Core 9 Web API project with SignalR hub endpoints (`DuelHub.cs`).
- Connect Godot C# client via `Microsoft.AspNetCore.SignalR.Client` NuGet package.
- Synchronize real-time state broadcasts: client sends actions (``PlayMana``, ``Attack``), server validates and broadcasts updated ``DuelGameState``.

### 5 Phase 5: Visual Polish, Shaders & Sound Design (Master Duel Style)

- Add Godot 4 custom fragment shaders for Shield Break disintegration & screen shake.
- Integrate civilization-specific attack beam visual effects (Fire flames, Water sapphire wave, Light laser ray).
- Add dynamic UI sound effects for card draw, tapping, shield shatter, and direct attack victory banner.